

## Nine useful commands

### **/help**

Show available commands and help.

/help

### **/model**

View or change the Claude model.

/model

### **/context**

Show what is using the context window.

/context

### **/compact**

Summarise the conversation to free context space.

/compact Keep the agreed requirements

### **/clear**

Start a fresh conversation in this terminal.

/clear

### **/resume**

Reopen an earlier Claude Code conversation.

/resume

### **/permissions**

Review or change tool permissions.

/permissions

### **/memory**

Open Claude Code memory files.

/memory

### **! command**

Run Bash without leaving Claude Code.

! git status

## Essential controls

Ctrl+C: interrupt; again at idle prompt: exit. Ctrl+D or /exit: exit.

Esc: interrupt. Esc twice: rewind options. Ctrl+Z: avoid while learning.

@path: mention a file. /btw: side question. Ctrl+B: background task.

## Scheduling in Claude Code

Works only while the session is open; scheduled tasks expire after seven days.

```
/loop 5m check whether the tests have finished
```

```
/schedule review open pull requests every weekday at 9am
```

Ask Claude in ordinary language to list or cancel scheduled tasks.

## Common commands

### pwd

Show the current folder.

```
pwd
```

### ls

List files; -l adds detail and -a includes hidden files.

```
ls -la
```

### cd

Change to another folder.

```
cd docs
```

### cd -

Return to the directory you were in immediately before.

```
cd -
```

### mkdir

Create a new folder.

```
mkdir outputs
```

### touch

Create an empty file or update an existing file's timestamp.

```
touch notes.txt
```

### echo

Display text or send it into a file using > or >>.

```
echo "Hello"
```

### cat

Display a small text file.

```
cat README.md
```

### grep

Search for text inside a file.

```
grep -n "premium" report.md
```

### code

Open a file or folder in VS Code.

```
code .
```

### nano

Open a simple terminal text editor.

```
nano notes.txt
```

### cp

Copy a file; use -r to copy a folder.

```
cp report.md backup.md
```

### mv

Move or rename a file or folder.

```
mv draft.md final.md
```

### rm

Permanently delete a file.

```
rm old_notes.txt
```

### pushd

Save the current directory and move to another one.

```
pushd docs
```

### popd

Return to the directory saved by pushd.

```
popd
```

### dirs -v

Show the numbered directory stack used by pushd/popd.

```
dirs -v
```

## Creating files, navigation and paths

touch creates an empty file; echo writes text when used with > or >>.

cd - switches back to the directory you just left.

Put quotes around spaces: cd "My Documents/project"

Use / in Git Bash paths. If nano is unavailable, use code filename.

**Copy terminal commands from the Markdown file, not from this PDF.**

## DANGER: rm -rf folder\_name

Permanently deletes the folder and everything inside it.

It does not use the Recycle Bin.

First run pwd and ls; then check the exact folder name.

Never use it with a blank, uncertain, broad or copied path.

## Get and check

### **git clone URL**

Download a repository into a new folder, usually once.

```
git clone https://github.com/example/project.git
```

### **git status**

Show changed, staged and untracked files.

```
git status
```

### **git remote -v**

Show the remote repository addresses.

```
git remote -v
```

### **git branch --show-current**

Show the name of the current branch.

```
git branch --show-current
```

### **git pull**

Fetch, then integrate by merge or configured rebase.

```
git pull
```

## Inspect and integrate

### **git fetch**

Download remote information without changing working files.

```
git fetch
```

### **git log HEAD..origin/main --oneline**

Inspect new remote commits before integrating.

```
git log HEAD..origin/main --oneline
```

### **git diff HEAD..origin/main**

Inspect remote line changes before integrating.

```
git diff HEAD..origin/main
```

### **git merge origin/main**

Integrate fetched remote main after inspection.

```
git merge origin/main
```

### **git diff**

Review unstaged line-by-line changes.

```
git diff
```

## Save and publish

### **git add FILE**

Stage one selected file for the next commit.

```
git add report.md
```

### **git commit -m "message"**

Record staged changes with a meaningful message.

```
git commit -m "Update report"
```

### **git log --oneline**

Show a compact commit history.

```
git log --oneline -10
```

### **git switch -c NAME**

Create a new branch and switch to it.

```
git switch -c update-report
```

### **git push**

Publish commits to the tracked remote branch.

```
git push
```

## Inspect before integrating

```
git fetch
```

```
git log HEAD..origin/main --oneline
```

```
git diff HEAD..origin/main
```

```
git merge origin/main
```

Replace main if the remote branch has a different name.

## First push

1. Check: `git branch --show-current`

2. If it is main: `git push -u origin main`

3. Otherwise, replace main with the actual branch name.

4. Later, plain `git push` is normally sufficient.